



THE UNIVERSITY OF
SYDNEY

ELEC5304

Project 2

Classification Network

Student(s):

Dawod Ghifari 520140154

Hilal Chamtie 540749283

Darin Li 500048292

Akshar Hossain 540823996

Teacher:

Luping Zhou

May 4, 2026

Contents

1	Introduction	2
2	Dataset Preparation	3
2.1	Splitting	3
2.2	Augmentation	3
2.3	Loading	4
3	Model Design and Training Strategy	5
3.1	Model A: ResNet-20	5
3.2	Model B: FastCIFARNet	6
4	Experimental Results	9
4.1	Objective (i): Model A, ResNet-20	9
4.2	Objective (ii): Model B, FastCIFARNet	9
4.3	Per-class Accuracy and Confusion Matrices	10
4.4	Comparison and Discussion	12
5	Task 2: Architectures for Limited Training Time	14
6	Conclusion	14

1 Introduction

In the past, access to high-quality training data was limited and image classification models were far less advanced. Researchers tested their models on disparate datasets, making cross-comparison of performance difficult. In 2009, Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton at the University of Toronto introduced a standardised benchmark known as CIFAR-10 [1]. The dataset was constructed by collecting natural images from the web and downsampling each to 32×32 pixels across 10 mutually exclusive classes, with 60,000 images in total [2, 3]. Its release allowed researchers worldwide to compare model performance on a common footing and accelerated progress in image recognition.

Building on CIFAR-10, this project develops a classification network that predicts the class label of a given input image. The report covers each stage of the pipeline, from data preprocessing to two distinct training strategies, and demonstrates how each design satisfies the following objectives:

- **Objective (i):** Design a network that achieves greater than 80% test accuracy.
- **Objective (ii):** Optimise a network that achieves at least 70% test accuracy within one minute of training on a T4 GPU.

2 Dataset Preparation

Before training, the raw image data must be arranged into a format suitable for the model. An intuitive analogy is cooking: ingredients (data) must be sorted, prepared, and delivered to the chef (the network) in measured portions. The three corresponding stages, *splitting*, *augmenting*, and *loading*, are described below.

2.1 Splitting

The original CIFAR-10 release ships with 50,000 training images and 10,000 test images. We combined all 60,000 images and applied a true 80/20 split, allocating 48,000 images to a training pool and 12,000 to a held-out test set. The training pool was further divided 90/10 into 43,200 training images and 4,800 validation images. The validation set acts as a practice exam used to monitor generalisation during training, while the test set is touched only once at the end to give an unbiased estimate of final performance. The split is summarised in Table 1.

Table 1: CIFAR-10 dataset split used in this project.

Subset	Images	Proportion of Total
Training	43,200	72%
Validation	4,800	8%
Test (held-out)	12,000	20%
Total	60,000	100%

2.2 Augmentation

Data augmentation applies stochastic transformations to training images so the network does not overfit to spurious cues such as object position or orientation. The transformations used are:

- **RandomCrop** (with 4-pixel padding): the image is padded to 40×40 and a 32×32 patch is randomly cropped, encouraging the model to be invariant to translations [4].
- **RandomHorizontalFlip**: each image is flipped along the vertical axis with probability 0.5, providing invariance to left/right orientation.
- **Normalisation**: pixel values are rescaled to a small range centred on zero, allowing optimisation to proceed more efficiently [5]. For a pixel value x , channel mean μ , and channel standard deviation σ ,

$$\hat{x} = \frac{x - \mu}{\sigma}. \quad (1)$$

The CIFAR-10 channel statistics $\mu = (0.4914, 0.4822, 0.4465)$ and $\sigma = (0.2470, 0.2435, 0.2616)$ used here are computed once over the training pool.

Augmentation is applied only to training data. Validation and test images receive normalisation alone, so that the reported metrics reflect the true generalisation of the model rather than an artefact of stochastic augmentation at evaluation time.

2.3 Loading

Because the full training set cannot fit in GPU memory at once, data is consumed in mini-batches of 128 images. Each epoch shuffles the training indices to remove ordering effects. The DataLoader is configured with `pin_memory=True` for faster host-to-device transfer and `persistent_workers=True` to avoid the cost of respawning worker processes between epochs. The DataLoader settings are summarised in Table 2.

Table 2: DataLoader configuration.

Setting	Value
Batch size	128
Shuffle (training)	True
<code>num_workers</code>	2
<code>pin_memory</code>	True
<code>persistent_workers</code>	True

3 Model Design and Training Strategy

A common challenge in training deep networks is the *vanishing gradient problem*: error signals propagated backward through many layers shrink at each step, so early layers receive almost no learning signal. A widely used remedy is the *residual connection*, which adds the input of a block to its output:

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}, \quad (2)$$

where $\mathbf{x}$ is the block input, $F(\mathbf{x})$ is the residual mapping the block must learn, and $H(\mathbf{x})$ is the desired output. This shortcut path lets gradients flow directly to earlier layers, improving optimisation in deep networks.

Both models in this report are convolutional neural networks (CNNs), but they differ sharply in depth, optimiser, and training schedule because each is tuned for a different objective. Model A (Section 3.1) uses residual connections to maximise final accuracy without a time constraint. Model B (Section 3.2) drops residual connections in favour of a wider, shallower design that converges in only three epochs under a strict 60-second budget.

3.1 Model A: ResNet-20

ResNet-20 is a compact residual CNN designed for CIFAR-10. With only $\sim 0.27\text{M}$ parameters and 20 layers, it is well suited to long, stable training and runs comfortably on Apple MPS or a Colab T4 GPU. The architecture begins with a stem convolution that maps the 3-channel RGB input to 16 feature maps, followed by Batch Normalisation and a ReLU activation:

- **Batch Normalisation** rescales the outputs of a layer to have approximately zero mean and unit variance over each mini-batch, stabilising the input distribution seen by the next layer and enabling higher learning rates.
- **ReLU** (Rectified Linear Unit), defined as $\text{ReLU}(z) = \max(0, z)$, introduces non-linearity by zeroing negative activations while leaving positive values unchanged, allowing the network to model complex relationships beyond linear transformations.

After the stem, three stages of residual *BasicBlocks* progressively increase channel depth across widths $\{16, 32, 64\}$. Each stage halves the spatial resolution at its first block via stride-2 convolution. A global average pooling layer collapses the spatial dimensions to a vector, which is passed to a single linear classifier producing 10 class logits. The high-level structure is summarised in Table 3.

Table 3: ResNet-20 architecture overview.

Stage	Output size	Channels	Blocks
Stem (Conv 3×3 + BN + ReLU)	32×32	16	1
Stage 1 (BasicBlock $\times 3$)	32×32	16	3
Stage 2 (BasicBlock $\times 3$, stride 2 first)	16×16	32	3
Stage 3 (BasicBlock $\times 3$, stride 2 first)	8×8	64	3
Global Average Pool	1×1	64	—
Linear (FC) classifier	—	10	—

The model is trained with Stochastic Gradient Descent (SGD) using Nesterov momentum $\beta = 0.9$ and weight decay 5×10^{-4} . SGD updates the weights in the direction that reduces the loss; Nesterov momentum looks ahead along the current velocity to achieve more stable convergence; weight decay penalises large weights to combat overfitting. A cosine-annealing schedule decays the learning rate from $\eta_{\max} = 0.1$ to 0 over 50 epochs, providing larger updates early in training and finer adjustments toward the end:

$$\eta_t = \frac{1}{2} \eta_{\max} \left(1 + \cos \frac{\pi t}{T} \right), \quad t = 0, 1, \dots, T. \quad (3)$$

Given sufficient training time, SGD with momentum is the canonical recipe for ResNets and tends to find sharper, well-generalising minima than adaptive methods. Since Model A operates without a time constraint, this is the natural choice. Hyperparameters are summarised in Table 4.

Table 4: Model A training configuration.

Hyperparameter	Value
Optimiser	SGD with Nesterov momentum
Momentum β	0.9
Weight decay	5×10^{-4}
Initial learning rate $\eta_{\max}$	0.1
Schedule	Cosine annealing to 0
Epochs	50
Batch size	128
Loss	Cross-entropy

3.2 Model B: FastCIFARNet

Model B faces a hard 60-second wall-clock training budget on a T4 GPU. On Colab, the CPU-bound data pipeline puts a floor of roughly 16 to 18 seconds on each full epoch regardless of model size. This caps the achievable epoch count at three, leaving roughly $\sim 1,000$ update steps in which to learn. SGD does not converge meaningfully in three epochs, so the long-training recipe used for Model A is no longer viable. Every design choice in Model B is therefore aimed at a single goal: extracting as much accuracy as possible from those $\sim 1,000$ steps.

Architecture. FastCIFARNet is a plain three-stage CNN with $\sim 0.98\text{M}$ parameters and width 48. Each stage consists of two $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}$ layers, with the second layer of each stage halving the spatial resolution. Channel counts double across stages, going from 48 to 96 to 192. A global average pool is followed by a single linear classifier. Skip connections are intentionally omitted: their main benefit appears over many epochs, and the additional memory traffic at each step would slow down a budget-bound run. To compensate, the network is made wider (more channels per layer), which extracts more information per gradient update and partially offsets the short training horizon. Table 5 summarises the architecture.

Table 5: FastCIFARNet architecture overview.

Stage	Output size	Channels	Layers
Stage 1 (Conv + BN + ReLU $\times$ 2, stride 2 on second)	16×16	48	2
Stage 2 (Conv + BN + ReLU $\times$ 2, stride 2 on second)	8×8	96	2
Stage 3 (Conv + BN + ReLU $\times$ 2, stride 2 on second)	4×4	192	2
Global Average Pool	1×1	192	—
Linear (FC) classifier	—	10	—

Training recipe. The training configuration is engineered for fast convergence:

- **AdamW** instead of SGD: adaptive per-parameter learning rates yield in one epoch what SGD typically needs ten epochs to achieve. Maximum learning rate $\eta_{\max} = 0.02$, weight decay 1×10^{-4} .
- **OneCycleLR**: the learning rate ramps up over the first 20% of training (`pct_start` = 0.2) and then anneals smoothly to a much smaller value, finishing inside the time budget without truncating the schedule.
- **Mixed-precision training** (AMP, `float16`): roughly halves the per-step compute cost on a T4 with little to no accuracy loss.
- **Label smoothing** ($\varepsilon = 0.1$): softens the one-hot target so the model does not become over-confident on its own predictions in only three epochs. The smoothed target distribution is

$$y_k^{\text{LS}} = (1 - \varepsilon) y_k + \frac{\varepsilon}{K}, \quad (4)$$

where y_k is the original one-hot label and $K = 10$ is the number of classes.

- **60-second hard budget**: training halts automatically once wall-clock time crosses 60 s, even mid-epoch.

The full configuration is reproduced in Table 6.

Table 6: Model B training configuration.

Hyperparameter	Value
Optimiser	AdamW
Maximum learning rate $\eta_{\max}$	0.02
Weight decay	1×10^{-4}
Schedule	OneCycleLR (<code>pct_start</code> = 0.2)
Mixed precision	AMP (<code>float16</code>)
Label smoothing ε	0.1
Epochs	3
Batch size	128
Wall-clock budget	60 s

A side-by-side summary of the two models is given in Table 7.

Table 7: Comparison of design choices for Model A and Model B.

	Model A (ResNet-20)	Model B (FastCIFARNet)
Objective	Maximise accuracy	$\geq 70\%$ in 60 s
Architecture	20-layer residual CNN	3-stage plain CNN
Width (channels)	16 / 32 / 64	48 / 96 / 192
Parameters	$\sim 0.27\text{M}$	$\sim 0.98\text{M}$
Skip connections	Yes	No
Optimiser	SGD + Nesterov	AdamW
Learning-rate schedule	Cosine annealing	OneCycleLR
Mixed precision	No	Yes (AMP)
Label smoothing	No	$\varepsilon = 0.1$
Epochs	50	3
Wall-clock budget	None	60 s

4 Experimental Results

4.1 Objective (i): Model A, ResNet-20

Model A was trained for 50 epochs as described in Section 3.1. Training progressed steadily: by epoch 10 the training accuracy reached 80.93% with a validation accuracy of 77.48%, and by epoch 30 the training accuracy had passed 89% (validation 84.62%). Final training and validation accuracies converged to 97.47% and 91.71% respectively at epoch 50, with a best validation accuracy of 91.79% at epoch 46. Training and validation curves are shown in Figure 1.

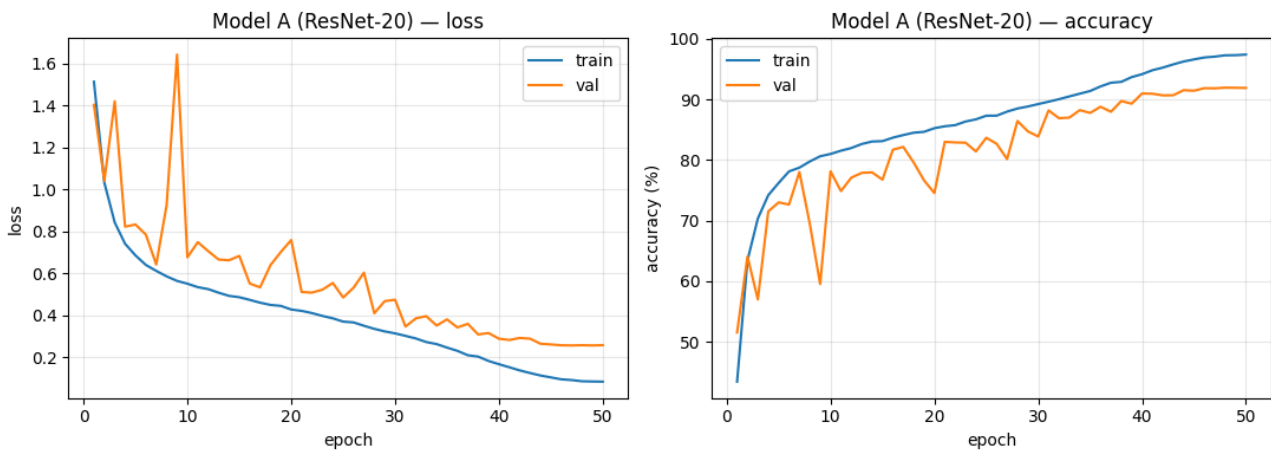


Figure 1: Model A (ResNet-20) training and validation curves over 50 epochs. Left: loss. Right: accuracy.

On the held-out test set, evaluated once at the end of training, Model A reached **91.31%** test accuracy, exceeding the 80% target by approximately 11 percentage points. The closeness of the best validation accuracy (91.79% at epoch 46), the final validation accuracy (91.71%), and the held-out test accuracy (91.31%) indicates that the model generalises well and shows no significant late-stage overfitting. Cosine annealing contributed visibly here: the gradual decay of the learning rate over the final 10 to 15 epochs allowed the model to settle into a sharp, well-generalising minimum rather than oscillate around it.

4.2 Objective (ii): Model B, FastCIFARNet

Model B was trained on a T4 GPU under a strict 60-second wall-clock budget, completing 3 epochs in 57.3 seconds. Validation accuracy improved sharply between epochs, reaching 56.56% after epoch 1, 70.25% after epoch 2, and 77.67% after epoch 3. Loss and accuracy curves over the three epochs are shown in Figure 2.

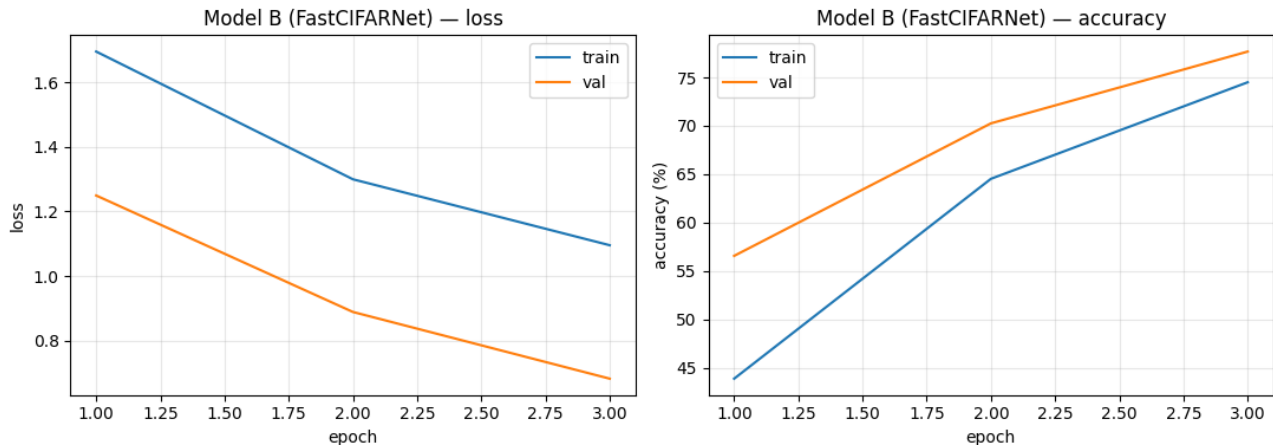


Figure 2: Model B (FastCIFARNet) training and validation curves over 3 epochs. Left: loss. Right: accuracy.

On the held-out test set, Model B achieved **77.09%** test accuracy, exceeding the 70% target by approximately 7 percentage points, with training completing in 57.3 seconds, comfortably inside the 60-second budget. The best validation accuracy (77.67% at epoch 3) and the held-out test accuracy (77.09%) are essentially identical, indicating that despite the very short training horizon the model did not overfit.

4.3 Per-class Accuracy and Confusion Matrices

Top-1 accuracy alone hides which classes each model finds difficult. Per-class recall and row-normalised confusion matrices on the held-out test set (12,000 images) are reported in Table 8 and Figures 3 and 4.

Table 8: Per-class test accuracy (recall) for both models, with test-set support per class.

Class	Support	Model A acc. (%)	Model B acc. (%)
airplane	1,197	91.65	82.71
automobile	1,167	95.80	89.72
bird	1,232	88.56	70.54
cat	1,244	80.47	56.35
deer	1,169	91.53	71.60
dog	1,206	86.90	65.84
frog	1,166	95.37	83.62
horse	1,247	92.86	77.63
ship	1,162	96.04	88.47
truck	1,210	94.71	86.12
Best	—	ship (96.04)	automobile (89.72)
Worst	—	cat (80.47)	cat (56.35)

Both models share the same hardest class, *cat*, and confuse it most often with *dog*, which is the canonical CIFAR-10 confusion pair. The pattern is consistent across the two models but markedly more pronounced in Model B: where Model A loses about 11 percentage points on

cat relative to its overall accuracy, Model B loses over 20 percentage points. This is consistent with the much shorter training horizon, which limits how finely Model B can separate visually similar animal classes. Vehicles (*automobile*, *ship*, *truck*) are the easiest classes for both models, reflecting the larger inter-class visual differences in that group.

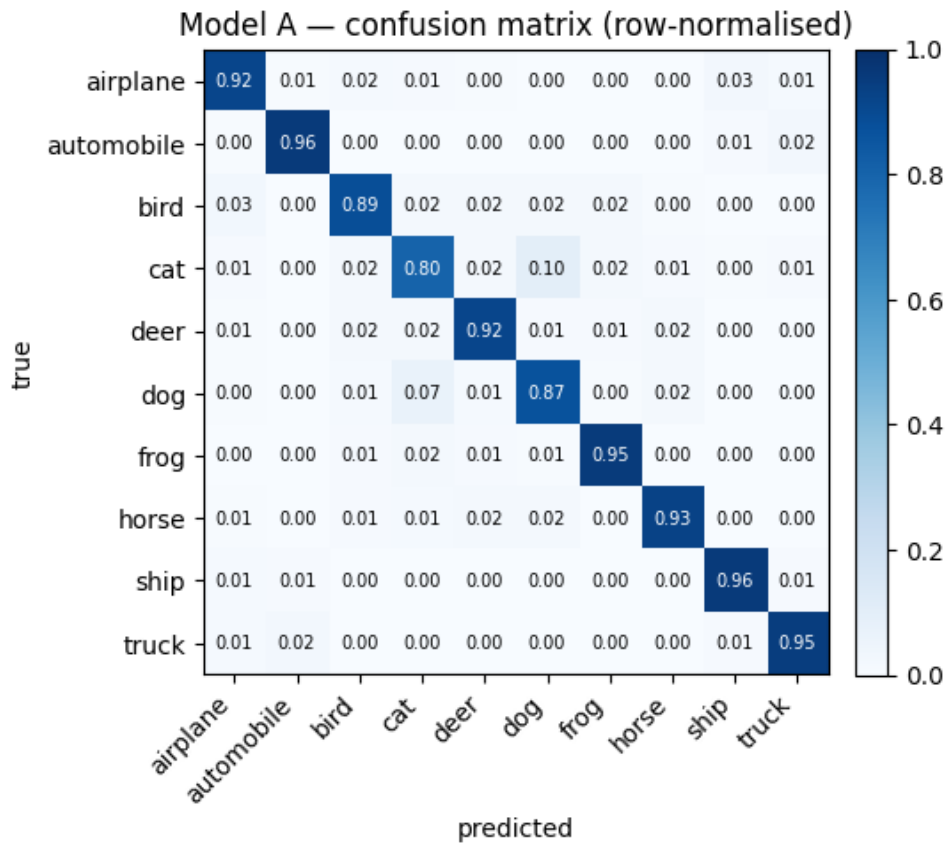


Figure 3: Model A (ResNet-20) row-normalised confusion matrix on the held-out test set.

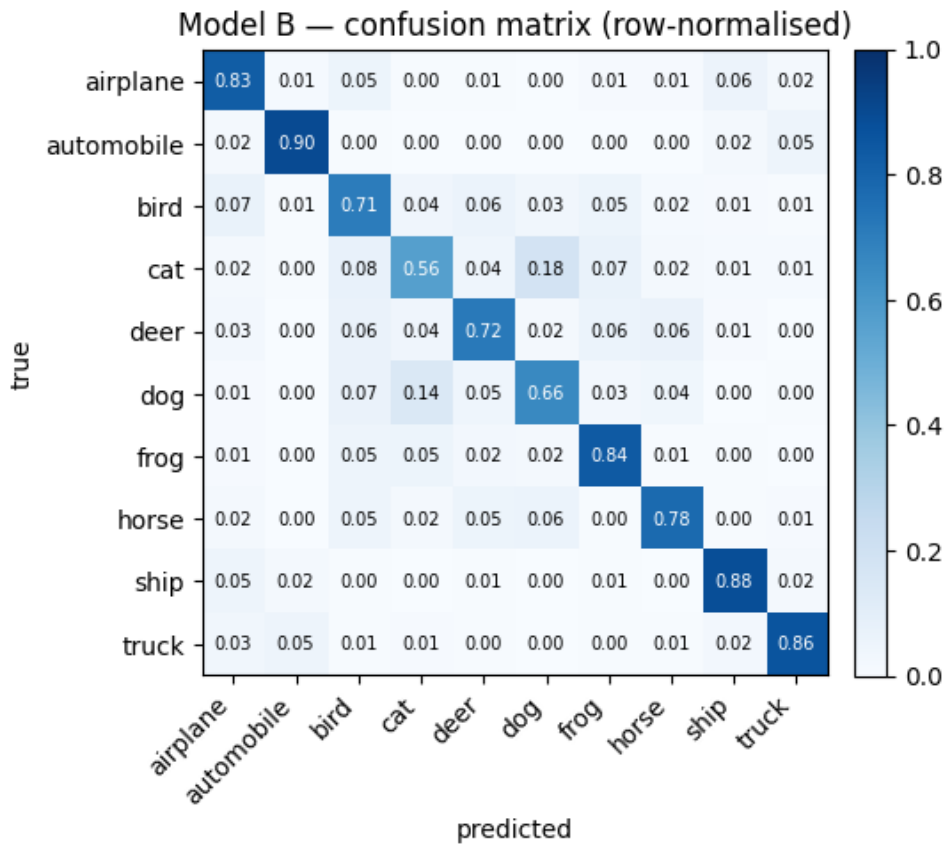


Figure 4: Model B (FastCIFARNet) row-normalised confusion matrix on the held-out test set.

4.4 Comparison and Discussion

The two models target different objectives, and this drives every architectural and optimisation choice. Model A is configured for maximum final accuracy, while Model B is configured for the highest accuracy attainable in 60 seconds. The headline outcomes are summarised in Table 9.

Table 9: Final results for both models on the held-out test set.

Model	Test acc. (%)	Target	Wall-clock	Margin over target
A (ResNet-20)	91.31	> 80%	— (50 epochs)	+11.31 pp
B (FastCIFARNet)	77.09	$\geq 70\%$ in 60 s	57.3 s	+7.09 pp

Model A produces stronger final performance because residual connections, SGD with momentum, and cosine annealing all scale well with training time. Under a 60-second limit, however, only about three epochs can be completed, which is far too few for this recipe to converge: ResNet-20 trained with SGD for three epochs would not reach the 70% threshold. Model B addresses this directly by trading depth and skip connections for width, and by replacing SGD with AdamW combined with OneCycleLR, AMP, and label smoothing, a combination explicitly tuned for fast early convergence. The cost is a lower ceiling: although Model B beats its own target by 7 percentage points, it sits roughly 14 percentage points below Model A’s final accuracy.

The fundamental trade-off is therefore between *ceiling* and *convergence rate*. When training time is plentiful, a deeper residual CNN trained with SGD reaches a higher final accuracy. When training time is the binding constraint, a shallower-but-wider CNN with adaptive optimisation and aggressive scheduling reaches a useful accuracy in a fraction of the time. Neither model dominates the other; each is the right answer for its own objective.

5 Task 2: Architectures for Limited Training Time

Q. What kind of neural network architecture is more suitable for learning under limited training time? Why?

When training time is the binding constraint, the most effective choice is typically a *compact, spatially biased CNN*, a network whose inductive biases already align with the task, so that useful features can be learnt from a small number of gradient updates. In our experiments, FastCIFARNet (a three-stage plain CNN with width 48 and approximately 1M parameters) reached 77.09% test accuracy in three epochs and under 58 seconds on a T4 GPU. The deeper ResNet-20 produced higher final accuracy when given 50 epochs, but did not converge fast enough to clear the 70% threshold inside a one-minute window.

There are three reasons a compact CNN suits this regime. First, convolutional layers encode strong priors that match the data: local connectivity captures local image structure, weight sharing reduces the number of parameters that have to be learnt, and translation equivariance means the model does not have to rediscover that an object's identity is independent of its position. Fully connected networks and Vision Transformers lack these built-in priors and need substantially more data and steps to learn the same spatial relationships. Second, Batch Normalisation makes optimisation tolerant of higher learning rates, which is a prerequisite for any fast-convergence schedule such as OneCycleLR. Third, a small parameter count makes each forward and backward pass cheap, so more useful update steps fit in the budget; widening rather than deepening the network (within reason) extracts more information per step without the latency cost of additional layers.

These structural choices were paired with optimisation choices that complement short-horizon training: AdamW for fast adaptation in the first epoch, OneCycleLR for a schedule that finishes cleanly inside the budget, mixed-precision training for cheaper compute per step, and label smoothing to suppress the over-confidence that a network can develop after only a thousand updates. The combined picture is that, under a strict training-time limit, a small spatially biased CNN paired with an adaptive, schedule-aware training recipe is more suitable than a larger or more general-purpose architecture, even if the larger architecture would ultimately reach a higher accuracy given more time.

6 Conclusion

This project developed two convolutional neural networks for CIFAR-10, each tuned to a different objective. Model A (ResNet-20) achieved 91.31% test accuracy over 50 epochs, comfortably exceeding the $> 80\%$ target, while Model B (FastCIFARNet) achieved 77.09% test accuracy in 57.3 seconds on a T4 GPU, exceeding the $\geq 70\%$ in-1-minute target. Together, the two models illustrate the central trade-off in practical deep learning: the architecture and training recipe that maximise final accuracy are not the same as those that maximise accuracy per unit of training time, and the right design depends on which side of that trade-off the application sits on.

References

- [1] B. Barz and J. Denzler, “Do We Train on Test Data? Purging CIFAR of Near-Duplicates,” *Journal of Imaging*, vol. 6, no. 6, p. 41, Jun. 2020. doi: 10.3390/jimaging6060041. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/34460587/>
- [2] “Comparative Analysis of CIFAR-10 Image Classification: Transfer Learning vs. User-Defined CNNs,” *The Startup*, Medium. [Online]. Available: <https://medium.com/swlh/comparative-analysis-of-cifar-10-image-classification-transfer-learning-vs-user-defined-cnns>
- [3] “Deep Learning with CIFAR-10 Image Classification,” *Towards Data Science*. [Online]. Available: <https://towardsdatascience.com/deep-learning-with-cifar-10-image-classification-64ab92110d79/>
- [4] Keras, “RandomCrop layer,” Keras API Reference. [Online]. Available: https://keras.io/api/layers/preprocessing_layers/image_augmentation/random_crop/
- [5] “Image Normalization,” ScienceDirect Topics, Engineering. [Online]. Available: <https://www.sciencedirect.com/topics/engineering/image-normalization>